

Physical Design of 4-bit ALU using OpenLane and Sky130 PDK

1. Introduction

This project demonstrates the complete RTL-to-GDSII ASIC physical design flow of a 4-bit Arithmetic Logic Unit (ALU) using open-source EDA tools. The design was implemented using Verilog HDL and synthesized, placed, routed, and verified using OpenLane and the Sky130 Process Design Kit (PDK).

The project helps in understanding the practical backend VLSI physical design flow including:

- RTL Design
- Logic Synthesis
- Floorplanning
- Placement
- Clock Tree Synthesis (CTS)
- Routing
- Static Timing Analysis (STA)
- Power Analysis
- DRC/LVS Verification
- GDSII Generation

2. Objective

The objective of this project is:

- To design a synchronous 4-bit ALU using Verilog HDL.
- To understand the ASIC physical design flow.
- To implement the RTL design using OpenLane.
- To analyze timing, area, power, and layout reports.
- To generate a final manufacturable GDSII layout.

3. Tools and Technologies Used

Tool/Technology	Purpose
Ubuntu (WSL2)	Linux environment
Docker	Containerized OpenLane environment

Tool/Technology	Purpose
OpenLane	Complete ASIC RTL-to-GDSII flow
OpenROAD	Physical design engine
Magic VLSI	Layout visualization
Verilog HDL	RTL design language
Sky130A PDK	Standard cell and technology library
Tcl	Automation and scripting

4. System Requirements

Component	Specification
OS	Windows 11 with WSL2 Ubuntu
RAM	16 GB
Storage	Minimum 20 GB free space
Docker	Docker Desktop installed

5. Project Description

The ALU performs four arithmetic and logical operations based on the select signal:

Select Signal (sel)	Operation
00	Addition
01	Subtraction
10	Bitwise AND
11	Bitwise OR

The design is synchronous and updates output on the positive edge of the clock.

6. RTL Design

Verilog Code

```
module alu (  
    input clk,  
    input [3:0] a,  
    input [3:0] b,  
    input [1:0] sel,  
    output reg [3:0] y  
);  
  
always @(posedge clk) begin  
    case(sel)  
        2'b00: y <= a + b;  
        2'b01: y <= a - b;  
        2'b10: y <= a & b;  
        2'b11: y <= a | b;  
    endcase  
end  
  
endmodule
```

7. OpenLane Configuration

config.json

```
{  
    "DESIGN_NAME": "alu",  
    "VERILOG_FILES": "dir::src/alu.v",  
    "CLOCK_PORT": "clk",  
    "CLOCK_PERIOD": 10,  
    "FP_CORE_UTIL": 30,  
    "PDK": "sky130A",  
    "STD_CELL_LIBRARY": "sky130_fd_sc_hd"  
}
```

8. Design Flow Explanation

8.1 RTL Design

The ALU functionality was described using Verilog HDL.

8.2 Synthesis

The RTL was converted into gate-level netlist using standard cells from Sky130 library.

Generated standard cells include:

- AND gates
 - OR gates
 - XOR gates
 - NAND gates
 - D Flip-Flops
 - Inverters
-

8.3 Floorplanning

The chip dimensions, core area, and power grid were defined.

Purpose:

- Define chip area
 - Allocate standard cell placement region
 - Generate power distribution network
-

8.4 Placement

Standard cells were physically placed inside the core area.

Goals:

- Reduce wirelength
 - Improve timing
 - Optimize area utilization
-

8.5 Clock Tree Synthesis (CTS)

Clock buffers were inserted to distribute the clock signal uniformly.

Purpose:

- Minimize clock skew
 - Improve timing reliability
-

8.6 Routing

Metal wires were generated to connect all placed standard cells.

Routing includes:

- Global Routing
 - Detailed Routing
-

8.7 Static Timing Analysis (STA)

Timing checks were performed to verify:

- Setup timing
 - Hold timing
 - Slack values
-

8.8 Signoff Checks

Final verification stages included:

- DRC (Design Rule Check)
 - LVS (Layout vs Schematic)
 - Antenna checks
 - IR Drop analysis
-

9. Commands Used

Clone OpenLane

```
git clone https://github.com/The-OpenROAD-Project/OpenLane.git
```

Install Dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install git curl wget build-essential -y
sudo apt install tcl -y
sudo apt install tcllib -y
sudo apt install python3-venv -y
```

Run OpenLane Container

```
docker run -it --rm
-e DISPLAY=$DISPLAY
-v /tmp/.X11-unix:/tmp/.X11-unix
-v $PWD:/openlane
-v ~/.ciel:/root/.ciel
-e PDK_ROOT=/root/.ciel
ghcr.io/the-openroad-project/openlane:latest
```

Run ASIC Flow

```
./flow.tcl -design alu
```

10. Results Obtained

10.1 Timing Report

Parameter	Result
TNS	0.00
WNS	0.00

Parameter	Result
Setup Slack	6.15 ns
Hold Slack	1.90 ns

Observation:

- No setup violations
- No hold violations
- Timing closure achieved successfully

10.2 Area Report

Parameter	Value
Total Cells	38
Chip Area	420.403200

Important Standard Cells Used:

- sky130_fd_sc_hd_dfxtp_2
- sky130_fd_sc_hd_xor2_2
- sky130_fd_sc_hd_and2_2
- sky130_fd_sc_hd_or2_2
- sky130_fd_sc_hd_nand2_2

Observation:

- 4 D Flip-Flops were inferred for 4-bit registered output.

10.3 Power Report

Component	Power
Sequential	2.39e-05 W
Combinational	1.49e-05 W
Total Power	3.88e-05 W

Observation:

- Sequential logic consumed higher power due to clocked flip-flops.

11. Generated Outputs

Generated files include:

File Type	Description
.v	Synthesized netlist
.def	Physical placement information
.lef	Abstract physical layout
.gds	Final ASIC layout
.sdf	Timing delay information

Final GDSII File:

alu.gds

12. Layout Visualization

The final layout was visualized using Magic VLSI.

Observed Features:

- Standard cell rows
- Metal routing layers
- Power rails
- Clock routing
- Physical placement of cells

13. Challenges Faced and Debugging Process

During implementation, several practical issues were encountered while setting up and running the ASIC flow. Debugging these issues provided valuable understanding of Linux environments, Docker integration, Tcl dependencies, and OpenLane infrastructure.

13.1 Python Virtual Environment Error

Error

```
The virtual environment was not created successfully because ensurepip is not available.
```

Cause

The required Python virtual environment package was missing in Ubuntu.

Fix

Installed Python venv package:

```
sudo apt install python3-venv -y
```

Learning

OpenLane internally creates Python virtual environments for dependency management.

13.2 Tcl JSON Package Missing

Error

```
can't find package json
```

Cause

OpenLane Tcl scripts required JSON parsing support which was not installed.

Fix

Installed Tcl library package:

```
sudo apt install tcllib -y
```

Learning

Tcl is heavily used in EDA tools and additional Tcl packages are often required.

13.3 Deprecated OpenLane Command

Error

```
[ERROR]: -from and -to are no longer supported.
```

Cause

Newer OpenLane versions no longer support older stage-specific command syntax.

Fix

Instead of:

```
./flow.tcl -design alu -to synthesis
```

Used:

```
./flow.tcl -design alu
```

Learning

EDA tools evolve frequently and command syntax may change between versions.

13.4 Container Manifest Version Mismatch

Error

```
[ERROR]: Container manifest not found.
```

Cause

Mismatch between local OpenLane scripts and Docker container version.

Fix

Used latest OpenLane Docker container directly:

```
docker run -it --rm
-e DISPLAY=$DISPLAY
-v /tmp/.X11-unix:/tmp/.X11-unix
-v $PWD:/openlane
-v ~/.ciel:/root/.ciel
-e PDK_ROOT=/root/.ciel
ghcr.io/the-openroad-project/openlane:latest
```

Learning

Containerized flows help maintain reproducible ASIC environments.

13.5 Docker Not Accessible Inside WSL

Error

The command 'docker' could not be found in this WSL 2 distro.

Cause

Docker Desktop WSL integration was disabled for Ubuntu.

Fix

Enabled:

Docker Desktop → Settings → Resources → WSL Integration → Ubuntu

Learning

WSL integration is required for Linux distributions to communicate with Docker engine.

13.6 Missing PDK Configuration

Error

[ERROR]: PDK is not specified.

Cause

PDK information was not included in config.json.

Fix

Added:

```
"PDK": "sky130A",  
"STD_CELL_LIBRARY": "sky130_fd_sc_hd"
```

Learning

ASIC flows require technology-specific libraries and process configuration.

13.7 Magic GUI Display Error

Error

```
no display name and no $DISPLAY environment variable
```

Cause

GUI display variables were not forwarded into Docker container.

Fix

Started Docker container with GUI forwarding support:

```
docker run -it --rm  
-e DISPLAY=$DISPLAY  
-v /tmp/.X11-unix:/tmp/.X11-unix  
-v $PWD:/openlane  
-v ~/.ciel:/root/.ciel  
-e PDK_ROOT=/root/.ciel  
ghcr.io/the-openroad-project/openlane:latest
```

Learning

GUI-based Linux applications inside containers require X11 display forwarding.

13.8 OpenLane Warnings During Flow

Warnings Observed

[WARNING]: Current core area is too small for the power grid settings chosen.

[WARNING]: VSRC_LOC_FILES is not defined.

Explanation

These warnings are common in small beginner designs and did not affect successful completion of the flow.

Learning

Not all warnings indicate design failure. Understanding warning severity is important in ASIC implementation.

14. Learning Outcomes

Through this project, the following concepts were learned:

- RTL design using Verilog
 - ASIC backend physical design flow
 - Synthesis and gate-level netlists
 - Placement and routing
 - Clock Tree Synthesis
 - Static Timing Analysis
 - Area and power analysis
 - Layout visualization
 - Docker and Linux workflow
 - Open-source ASIC toolchain usage
-

15. Applications

ALUs are fundamental blocks used in:

- CPUs
- Microcontrollers
- DSP processors
- Arithmetic accelerators

- Embedded systems
-

16. Conclusion

The project successfully implemented a 4-bit ALU using OpenLane and Sky130 PDK through the complete RTL-to-GDSII ASIC design flow. Timing closure was achieved successfully without setup or hold violations. The project provided practical understanding of physical design concepts including synthesis, placement, routing, timing analysis, and layout generation.

This project serves as a strong foundation for advanced VLSI physical design projects and backend ASIC implementation.

17. Future Improvements

Future enhancements can include:

- Larger ALU architecture
 - Pipelined ALU
 - Low-power optimization
 - Higher frequency constraints
 - Congestion optimization
 - Multi-clock domain design
 - Advanced floorplanning
-

18. Important File Locations

The following directories and files were generated during the OpenLane flow execution.

Project Root Directory

~/OpenLane/designs/alu

RTL Source File

```
~/OpenLane/designs/alu/src/alu.v
```

OpenLane Configuration File

```
~/OpenLane/designs/alu/config.json
```

Run Directory

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30
```

Note:

The RUN directory name changes automatically based on timestamp.

Logs Directory

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/logs
```

Contains:

- synthesis logs
 - placement logs
 - CTS logs
 - routing logs
 - signoff logs
-

Reports Directory

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/reports
```

Contains:

- timing reports
 - power reports
 - area reports
 - manufacturability reports
-

Results Directory

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/results
```

Contains generated outputs for every ASIC implementation stage.

Synthesized Netlist

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/results/synthesis/alu.v
```

Final GDSII File

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/results/final/gds/alu.gds
```

Magic Layout File

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/results/final/mag/alu.mag
```

DEF File

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/results/final/def/alu.def
```

LEF File

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/results/final/lef/alu.lef
```

Timing Reports

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/reports/synthesis/2-syn_sta.summary.rpt
```

Power Report

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/reports/synthesis/2-syn_sta.power.rpt
```

Area Report

```
~/OpenLane/designs/alu/runs/RUN_2026.05.07_13.40.30/reports/synthesis/1-synthesis.AREA_0.stat.rpt
```

19. Git and GitHub Workflow Guide

19.1 What is Git?

Git is a version control system used to:

- track project history
- manage file changes
- collaborate with teams
- maintain multiple project versions

Git is widely used in:

- VLSI design teams
- software engineering

- ASIC verification
 - RTL development
 - automation scripting
-

19.2 What is GitHub?

GitHub is a cloud platform used to store Git repositories online.

Benefits:

- project portfolio creation
 - team collaboration
 - backup of projects
 - version history tracking
 - open-source contribution
-

19.3 Common Git Workflow

Typical engineering workflow:

```
Modify files
↓
Git add
↓
Git commit
↓
Git push
```

19.4 Important Git Commands

Check Git Version

```
git --version
```

Initialize Repository

```
git init
```

Creates a new Git repository.

Rename Branch to Main

```
git branch -m main
```

Check Repository Status

```
git status
```

Shows:

- modified files
 - staged files
 - untracked files
-

Add Files to Git

```
git add .
```

Adds all current files for tracking.

To add specific file:

```
git add filename
```

Commit Changes

```
git commit -m "Commit message"
```

Creates a project checkpoint.

Example:

```
git commit -m "Added FIFO RTL and config"
```

Connect Local Repo to GitHub

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

Push Project to GitHub

```
git push -u origin main
```

Uploads local repository to GitHub.

Pull Latest Changes

```
git pull
```

Downloads latest changes from GitHub.

Clone Existing Repository

```
git clone REPOSITORY_LINK
```

Example:

```
git clone https://github.com/user/project.git
```

19.5 What is .gitignore?

`.gitignore` tells Git which files/folders should not be uploaded.

Common ignored files in VLSI projects:

```
runs/  
*.log  
*.tmp  
venv/
```

Reason:

- generated automatically
 - very large
 - unnecessary for repository
-

19.6 Recommended GitHub Repository Structure

```
README.md  
src/  
reports/  
screenshots/  
config.json
```

19.7 Recommended Files to Upload

Upload:

- RTL source code
- reports
- screenshots
- configuration files
- documentation

Do NOT upload:

- temporary tool files
 - large generated folders
 - Docker cache
 - virtual environments
-

19.8 GitHub Authentication

GitHub no longer supports password authentication for Git operations.

Instead, Personal Access Tokens (PAT) are used.

PAT steps:

1. Generate token from GitHub settings
2. Enable repo access
3. Use token as password during git push

19.9 Daily Git Workflow Example

Example workflow after modifying project:

```
cd project_folder

git status

git add .

git commit -m "Updated synthesis reports"

git push
```

19.10 Learning Outcomes from Git Usage

Through Git usage, the following concepts were learned:

- version control
- repository management
- project organization
- collaborative workflow
- GitHub authentication
- professional project publishing

20. References

1. OpenLane Documentation
2. OpenROAD Project
3. Sky130 PDK Documentation
4. Magic VLSI Documentation
5. Verilog HDL Reference